

From Overloaded to Guaranteed: High-Throughput Multi-SLO Enforcement for LoRA-Assisted On-Premise LLM Deployment

ZESHEN ZHANG, HAN ZHAO*, WEIHAO CUI, and QUAN CHEN, Shanghai Jiao Tong University, China

YU LIU and YONGJUN DENG, Ant Group, China

JING YANG, JIUCHEN SHI, CHEN CHEN, YOUMIN CHEN, YU FENG, and MINYI GUO, Shanghai Jiao Tong University, China

As Large Language Models (LLMs) become essential in privacy-sensitive sectors like hospitals and government agencies, the on-premise LLM servers offer a cost-effective and secure alternative to public cloud services. However, these resource-constrained servers struggle to guarantee heterogeneous Service Level Objectives (SLOs) when serving multiple LoRA-adapted services simultaneously. Existing serving frameworks suffer from severe SLO violations due to the computational overhead of LoRA layers and the rigid nature of batch scheduling. To address this, we propose HALO, a scheduling method tailored for LoRA-assisted on-premise LLM deployment. HALO introduces two key innovations: a spatial multiplexing strategy that overlaps Base and LoRA computations by partitioning GPU Streaming Multiprocessors (SMs), and an SLO-aware scheduler that decouples request execution based on "request-level slack." By prioritizing urgent tasks and utilizing idle budget for traffic shaping, HALO significantly mitigates resource contention. Our evaluation demonstrates that HALO minimizes SLO violations while improving throughput compared to state-of-the-art baselines.

CCS Concepts: • **Computer systems organization** → **Cloud Computing**.

Additional Key Words and Phrases: LLM system, LoRA, SLO

1 Introduction

As LLMs achieve state-of-the-art performance in many domains[9, 24, 35], the on-premise LLM deployment has become the preferred choice for sensitive sectors like hospital and government[10, 11, 23, 36]. Compared to public LLM services like ChatGPT, the on-premise deployment offers two advantages: cost efficiency and data privacy. For instance, a single server equipped with four NVIDIA A100 GPUs costs about \$150,000. In contrast, consuming the same volume of tokens via a public service can cost about \$1,500,000 per year[5, 25]. More importantly, using LLM services have risks of exposing proprietary data to third-party providers.

Figure 1 shows a hospital workflow served by such an on-premise LLM server. The server hosts four services, which share a frozen base model and use Low-Rank Adaptation (LoRA)[14] for task-specific optimization[6, 18, 30, 31]. These LoRA services have different Service Level Objectives (SLOs). For example, the Analysis Assistant requires a strict Time-to-First-Token (TTFT) of 1000ms for the prefill phase and Time-Between-Tokens (TBT) of 50ms for the decode phase. In contrast, the Intelligent Triage tolerates a looser latency budget (2000ms TTFT, 200ms TBT).

Given the strict SLOs inherent to these workloads, efficient scheduling is critical. However, current on-premise LLM servers rely on the default scheduling policies of mainstream frameworks (vLLM[16] and SGLang[42]), which lack support for multi-SLO scheduling. Consequently, we find

*Corresponding author.

Authors' Contact Information: Zeshen Zhang, gdeepolar@gmail.com; Han Zhao, zhao-han@cs.sjtu.edu.cn; Weihao Cui, weihao@sjtu.edu.cn; Quan Chen, chen-quan@cs.sjtu.edu.cn, Shanghai Jiao Tong University, Shanghai, China; Yu Liu, ly109106@antgroup.com; Yongjun Deng, dengyongjun.dyj@antgroup.com, Ant Group, China; Jing Yang, jyang23@gzu.edu.cn; Jiuchen Shi, shijiuchen@sjtu.edu.cn; Chen Chen, chen-chen@sjtu.edu.cn; Youmin Chen, chenyoumin@sjtu.edu.cn; Yu Feng, y-feng@sjtu.edu.cn; Minyi Guo, guo-my@cs.sjtu.edu.cn, Shanghai Jiao Tong University, Shanghai, China.

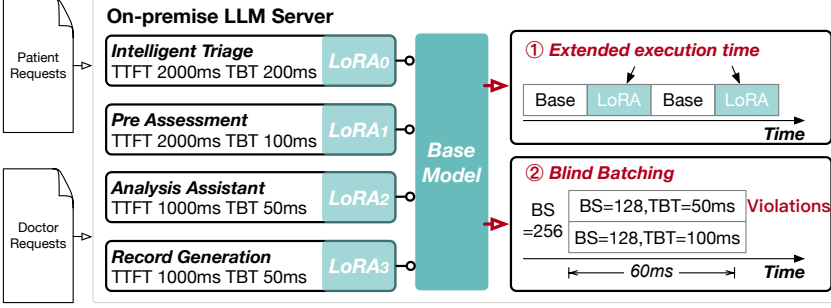


Fig. 1. The hospital workflow of an on-premise LLM server.

that on-premise LLM servers are highly susceptible to request overload within limited resources. This manifests as severe SLO violations and low system throughput. These performance issues stem from two fundamental causes: LoRA-induced throughput degradation and SLO violations driven by blind batching.

LoRA-induced throughput degradation. While existing frameworks adopt the chunked-prefill mode to improve system throughput, serving LoRAs still introduces non-negligible overhead. Specifically, chunked-prefill splits prefill into smaller chunks. Each chunk merges with the decode batch to form a mixed batch, improving the overall throughput. However, LoRA adds adapters on top of the model backbone and increases computation for both prefill and decode. Our results in §2.4.2 show that LoRAs bring an average 44.6% throughput drop. On resource-constrained on-premise hardware, this overhead intensifies resource contention and elevates SLO violations.

SLO-unaware blind batching. While batching is essential for GPU efficiency, it forces all requests in a batch to advance in lockstep regardless of their individual SLOs. For example, serving Llama3-8B[13] with LoRAs on an A100 GPU yields a decode latency of 47ms at batch size 384 and 57ms at batch size 512. If we set the maximum decode batch size for 50ms targets, 100ms-target requests in the decode batch easily block newly arrived 50ms-target requests from entering computation due to the limited request slots. This further causes TTFT violations. If we set the maximum batch size for 100ms requests, the 50ms target requests directly suffer TBT violations.

Although many works attempt to optimize SLO guarantees for LLM serving, they are insufficient for the constraints of on-premise servers. Specifically, Punica[6] reduces LoRA execution time via custom kernels, but it still suffers from sequential execution of LoRA adapters and base model operators. Meanwhile, VTC[32] and QoServe[12] focus on adjusting request serving order for inter-service fairness and SLO guarantees. Once a request enters computation, it does not leave the running batch. Therefore, no existing method addresses both the throughput degradation caused by LoRA and the SLO violations caused by blind batching.

Under these circumstances, we identify two opportunities to improve the SLO guarantee in the on-premise LLM server. First, we could mitigate LoRA overhead via spatial multiplexing. By partitioning Streaming Multiprocessors (SMs) between frozen backbone (base layer) and LoRA layer, we could enable the concurrent execution of two layers. This further helps improve hardware utilization and end-to-end latency.

Second, heterogeneous SLOs provide a resource dimension overlooked by existing work: temporal slack. Requests with relaxed SLOs such as TBT of 100ms have headroom relative to their deadlines in each decode iteration. When the actual decode latency is 50ms, a request with a 100ms TBT target can execute every two iterations without violating its SLO. This means that by breaking blind batching, requests with slack can yield compute resources to requests with urgent targets or

those facing SLO violations. This helps on-premise LLM serving increase acceptable concurrency and heterogeneous SLO guarantees under resource constraints.

However, capitalizing on these opportunities is non-trivial. First, determining the optimal SM partition involves a complex configuration space. The ideal split is highly sensitive to batch size, LoRA injection points (e.g., QKV vs. FFN), and hardware characteristics. This makes finding the ideal partition difficult. Second, as a request accumulates more slack, it could better improve concurrency. But once it exits, its accumulated slack drops to zero immediately. How to coordinate slack accumulation with attained throughput is a challenging problem. Furthermore, LLM inference has two stages. How to coordinate slack management across both stages to maximize system performance is another challenge.

To this end, we propose HALO, a scheduling method for LoRA-assisted on-premise LLM servers. HALO consists of a lightweight partition searcher and an SLO-aware stage-unified scheduler. The lightweight partition searcher finds the optimal SM partition for each base-LoRA layer pair with minimal overhead. The SLO-aware stage-unified scheduler jointly manages slack across both stages. It flexibly adjusts the compute allocation between decode and prefill within each decode iteration to maximize SLO guarantees and system throughput.

Specifically, during the serving framework initialization, the searcher automatically identifies the optimal SM partition for each base-LoRA layer pair under current hardware. To navigate the large configuration space, the searcher proposes a feedback-driven exploration strategy that minimizes the overhead. At runtime, the scheduler introduces a decoupling mechanism to maximize efficiency. It maintains an output token queue that releases generated tokens to the client precisely at their TTFT and TBT targets. This mechanism effectively acts as a traffic shaper, smoothing out bursts by buffering tokens generated during low-load periods and releasing them steadily at a high load.

Further, the scheduler introduces a novel metric called request-level slack. It unifies slack from prefill and decode and consolidates fragmented slack into longer time slices. By monitoring this slack, the scheduler identifies requests that can bypass some decode iterations without violating TBT targets. This helps prioritize urgent tasks facing SLO violations. Specifically, these urgent requests may be in the prefill stage or the decode stage. The scheduler jointly manages request scheduling across both stages to support both TTFT and TBT guarantees.

Our evaluation demonstrates that HALO effectively resolves resource contention in on-premise servers. Compared to state-of-the-art baselines, HALO improves P99 TTFT (99%-ile TTFT) by 2.1× to 5.0×, and achieves 19.5% to 29.5% throughput improvement. The main contributions of this paper are as follows:

- **Characterization of Resource Contention.** We identify that severe SLO violations in on-premise LLM servers stem from two primary sources: the computational overhead of LoRA layers and the blind batching mechanism.
- **Automated Spatial Multiplexing.** We propose a lightweight partition searcher that spatially partitions SMs between Base and LoRA layers. This enables concurrent execution that effectively hides LoRA latency overhead without manual tuning.
- **SLO-aware Stage-unified Scheduling.** We design a scheduler based on a novel request-level slack metric. This decouples request execution from batch processing, allowing the system to dynamically prioritize urgent tasks and utilize the latency budget of non-urgent requests.

2 Background and Motivation

In this section, we first provide background on LLM serving and the specific operational workflows of on-premise LLM servers. We then characterize the inefficiencies of state-of-the-art methods and identify key opportunities to address these challenges.

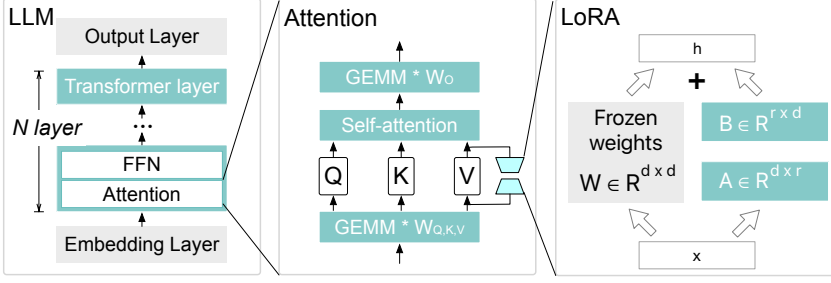


Fig. 2. The general architecture of an LLM model.

2.1 LLM Basics

2.1.1 LLM Model Structure. Modern large language models (LLMs), such as Llama[13] and Qwen[34], are primarily built upon the decode-only Transformer architecture. As shown in Figure 2, these models consist of a stack of identical transformer layers. Each transformer layer comprises two primary sub-layers: Self-Attention and a Feed-Forward Network (FFN).

In the attention layer, input hidden states are projected into Query, Key, and Value vectors via weight matrices (W_Q , W_K , W_V). The mechanism then computes attention scores to capture contextual dependencies between tokens, aggregating the results via an output projection (W_O). Subsequently, the FFN applies non-linear transformations, typically implemented as expansion (Gate/Up) and reduction (Down) projections.

2.1.2 LLM Inference. LLM inference consists of two phases: prefill and decode. The prefill phase processes the input prompt to generate the first token, while the decode phase generates subsequent tokens autoregressively. Performance is evaluated against two SLOs: Time-To-First-Token (TTFT) for the prefill and Time-Between-Tokens (TBT) for the decode.

While prefill is compute-bound, decode is memory-bound, easily suffering from low GPU utilization. To mitigate this, current systems (vLLM[16]/SGLang[42]) adopt chunked-prefill [1] as the default serving mode. It splits the prefill requests into smaller chunks. Each chunk merges with the decode batch to form a mixed batch for improving the GPU utilization and overall throughput.

2.1.3 Low-Rank Adaptation (LoRA). Low-Rank Adaptation is a parameter-efficient fine-tuning technique designed to adapt pre-trained LLMs to specific downstream tasks [14]. As shown in Figure 2, instead of updating the massive weight matrix $W \in \mathbb{R}^{d \times d}$, LoRA freezes W and introduces two low-rank matrices, $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$. The rank r is chosen to be significantly smaller than the weight dimensions d . The forward pass is modified such that the output is $h = Wx + BAx$. When serving a single LoRA, we can merge BA into W beforehand. However, when serving multiple LoRA adapters, weight merging is infeasible. The term BAx must be computed explicitly.

2.2 Why On-premise Deployment?

Benefitting from the rapidly improving capabilities of open-source models, on-premise LLM deployment has attracted significant industry attention. According to the IDC report[29], on-premise infrastructure accounted for 15.9% of global AI infrastructure spending in Q2 2025. This trend is rooted in two advantages: absolute data privacy and low operating costs.

Data privacy. On-premise deployment ensures that sensitive data never leaves the organization's own infrastructure, which is critical for hospitals, government agencies, and enterprises handling proprietary information.

Table 1. The representative LLM services in a hospital and their corresponding SLOs and datasets. TBT-1 is the TBT target for 7B and 8B LLMs, while TBT-2 is the TBT target for 30B LLMs. Following prior works [1, 28], 30B LLMs have looser TBT targets.

Services	TTFT	TBT-1	TBT-2	Dataset
Intelligent Triage	2000ms	200ms	200ms	ShareGPT [3]
Pre Assessment	2000ms	100ms	200ms	ChatDoctor [17]
Analysis Assistant	1000ms	50ms	100ms	AlpaCare [41]
Record Generation	1000ms	50ms	100ms	MedicalData [22]

Table 2. The representative LLM services in a service center and their corresponding SLOs and datasets.

Services	TTFT	TBT-1	TBT-2	Dataset
Service Triage	2000ms	200ms	200ms	ShareGPT [3]
Document Check	3000ms	100ms	200ms	Alpaca [33]
Realtime Translation	1000ms	50ms	100ms	WMT [4]
Information Retrieval	1000ms	50ms	100ms	AzureTrace [26]

Cost efficiency. Consider a mid-sized hospital running an on-premise LLM server at 15 requests/second for 12 hours/day, yielding roughly 648,000 requests/day. Assuming an average input length of 512 tokens and output length of 256 tokens, the annual token volume reaches 121B input tokens and 60.6B output tokens. At GPT-4o pricing (\$2.50/1M input tokens, \$10.00/1M output tokens), the equivalent public cloud cost totals \$900,000 per year. By contrast, a single 4×A100 server can sustain this workload (RPS≈15) at a one-time hardware cost of only \$150,000, which is less than one quarter of the annual cloud expenditure.

2.3 Multi-SLOs in On-premise LLM Servers

The on-premise LLM servers have become the preferred deployment solution for privacy-sensitive sectors [10, 11, 23, 36]. Table 1 and Table 2 illustrate representative services found in a hospital and a government service center.

As for the hospital scenario, there are four mainstream services: *Intelligent Triage*, *Pre Assessment*, *Analysis Assistant*, and *Record Generation*. The workflow follows the patient journey: a patient first interacts with the intelligent triage agent for routing, followed by a detailed pre-consultation assessment. Subsequently, the doctor leverages the analysis assistant for real-time diagnostic support and concludes by using the medical record generation.

As for the service center scenario, there are also four mainstream services: *Service Triage*, *Document Check*, *Realtime Translation*, and *Information Retrieval*. Citizens are first routed by the service triage agent, followed by document pre-screening to verify materials. Finally, clerks utilize translation and retrieval tools to facilitate the transaction.

Crucially, as human experts (doctors and clerks) constitute the critical path in these interactions, the services assisting them demand significantly stricter latency guarantees. As shown in Table 1, the *Analysis Assistant* requires a strict TBT of 50ms (with a 1000ms TTFT) to ensure conversational fluidity, whereas the automated *Intelligent Triage* tolerates a looser 200ms TBT (2000ms TTFT) [7, 20, 32]. To evaluate these scenarios realistically, we utilize datasets derived from actual medical and administrative scenarios.

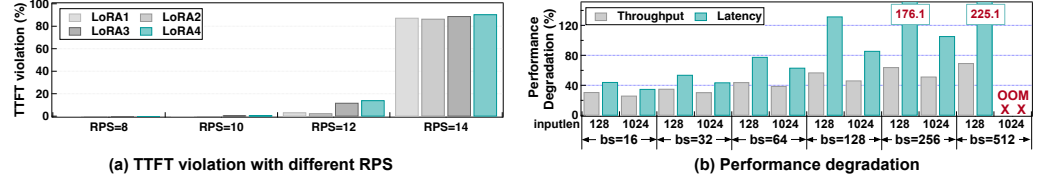


Fig. 3. (a) The TTFT violation of the current serving system under different workloads (requests per second, RPS). L1, L2, L3, and L4 represent four LoRA services, which are *Intelligent Triage*, *Pre Assessment*, *Analysis Assistant*, and *Record Generation*. (b) The performance degradation of decode latency and throughput under different running setups.

2.4 Inefficiencies

Since existing frameworks lack support for multi-SLO guarantees, current on-premise deployments set the maximum running batch size based on the strictest TBT target. Under the chunked-prefill mode, this maximum batch size limits the total size of the mixed batch (decode batch size + prefill chunk size). Meanwhile, because chunked-prefill prioritizes TBT over TTFT, existing systems rarely encounter TBT violations but mainly suffer from TTFT violations.

2.4.1 SLO violations. In this experiment, we configure the LLM server with a single NVIDIA A100 GPU to host the four hospital services. The system serves Llama3-8B using SGLang[42] in chunked-prefill mode. The maximum batch size is set to 384 based on the 50ms TBT target.

Figure 3(a) shows the TTFT violation rates across varying load levels (requests per second, RPS). As shown, services with strict SLOs begin to violate deadlines when the load exceeds 10 requests per second. After the load passes 12 requests per second, all services suffer significant violations. This demonstrates that the scheduling policies of existing frameworks fail to maintain SLOs under limited compute resources.

2.4.2 Root Causes. Our in-depth analysis attributes these SLO violations to two primary inefficiencies: sequential and inefficient LoRA execution, and SLO-agnostic blind batching.

Firstly, we collect the latency and throughput performance with four LoRA services enabled. Our results show that LoRA-enabled inference suffers an average 94.5% latency increase and 44.6% throughput degradation (Input Length: 128 and 1024, Output Length: 8). This is because the base layer and LoRA layer execute sequentially even though they have no data dependency. To address this problem, Punica employs specialized kernels to accelerate LoRA execution. Figure 3(b) shows the normalized performance of LoRA-enabled serving compared to the vanilla baseline. It still incurs an average 31.7% latency degradation and 23.8% throughput degradation. This indicates that LoRA serving with Punica[6] still suffers from significant performance loss.

Secondly, we show why existing systems suffer from SLO violations and why existing methods cannot solve this problem. Figure 4 shows the scheduling effects of the FCFS method and two state-of-the-art scheduling methods, VTC[32] and QoServe[12]. Specifically, VTC adjusts the request serving order to guarantee inter service fairness. This means multiple services process a fair number of tokens within a given time period. QoServe attempts to prioritize user-facing services and defer low-priority offline inference to optimize SLO guarantees. In addition, QoServe dynamically adjusts the maximum batch size based on the strictest user-facing service TBT target in the running batch to improve GPU utilization.

As shown in Figure 4, assume the maximum batch size on the current GPU is 4. The base time unit between two consecutive timestamps is 50ms. $R1$, $R2$, and $R3$ are currently in computation. $R1$

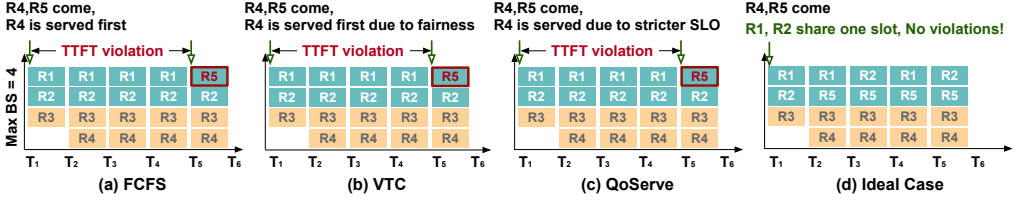


Fig. 4. The scheduling decisions and effects under FCFS, VTC, QoSServe and ideal case.

and R_2 belong to service *LoRA1* with a TTFT target of 200ms and a TBT target of 100ms. R_3 belongs to service *LoRA2* with a TTFT target of 100ms and a TBT target of 50ms. At time T_1 , requests R_4 and R_5 arrive. R_4 belongs to service *LoRA2* and R_5 belongs to service *LoRA1*.

For the FCFS method, when R_4 and R_5 arrive at time T_1 , only one running slot remains. R_4 is served while R_5 waits in the queue. Since LLM requests occupy a slot for an extended period due to the autoregressive nature of decoding, R_5 suffers a TTFT violation at time T_4 .

For the VTC method, since the dominant service in the current system is service *LoRA1*, the fairness calculation determines that R_4 from service *LoRA2* should be prioritized over R_5 . After R_4 is served, R_5 still suffers a TTFT violation at time T_4 .

For the QoSServe method, it adopts earliest deadline first strategy to schedule user facing services under high load. Since R_4 has a tighter TTFT target than R_5 , R_4 is served first. As a result, R_5 still suffers a TTFT violation at time T_4 . Note that, since all requests belong to user-facing services, the offline inference deferral proposed by QoSServe does not apply in the on-premise scenario.

In the ideal case, when R_4 and R_5 arrive, R_4 is served first. Since R_1 and R_2 both have a TBT target of 100ms and the current decode iteration takes 50ms, R_1 and R_2 can share one running slot by executing alternately. This frees up a slot so that R_5 can be served immediately. In this way, the TTFT violation for R_5 is eliminated.

Based on the above analysis, existing methods fail to provide SLO guarantees due to blind batching. Once a request enters the running batch, it cannot exit. Even if it has a relaxed SLO and could yield its running slot, it cannot help requests with urgent SLOs avoid violations.

2.5 Opportunities and Challenges

In a nutshell, while existing serving frameworks simplify multi-service deployment through batching, they suffer from the throughput degradation inherent to LoRA computation and the rigidity of SLO-agnostic scheduling. Facing these limitations on resource-constrained on-premise LLM servers, we identify two major opportunities to improve performance.

- We can reduce LoRA overhead using spatial multiplexing. By splitting the SMs between the base layer and the LoRA layer, we can run both components simultaneously. This parallelism aligns their completion times and improves the latency.
- Heterogeneous SLOs provide a resource dimension overlooked by existing work: temporal slack. Requests with relaxed SLOs have headroom relative to their deadlines in each decode iteration. This slack can be used to increase the concurrency capacity and improve overall SLO guarantees.

However, utilizing these two opportunities is non-trivial. First, implementing spatial multiplexing requires finding the optimal SM partition from a great search space. Finding the optimal one is challenging as it changes based on batch size and the specific LoRA layers used (e.g., QKV vs. FFN). Second, slack accumulation does not necessarily translate into throughput improvement. Once

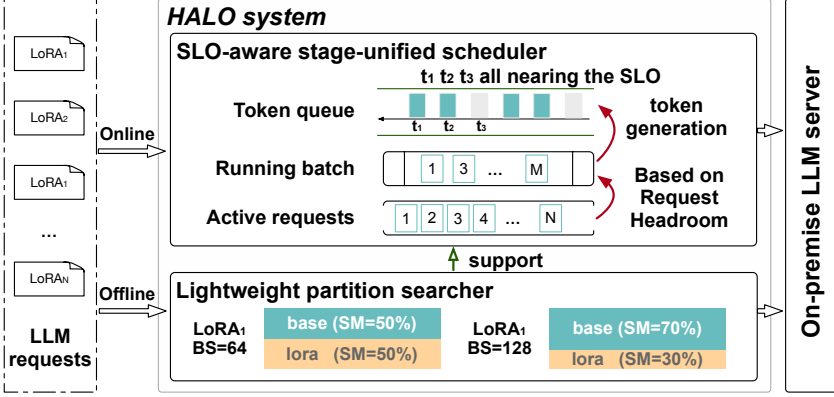


Fig. 5. The design overview of HALO.

a request exits, its accumulated slack becomes meaningless. How to coordinate slack accumulation with attained throughput is a challenging problem. Furthermore, how to coordinate slack management across both stages to maximize system performance is another challenge.

3 Design Overview

In this section, we present HALO, a scheduling method designed to enforce heterogeneous SLOs on resource-constrained on-premise LLM servers. As shown in Figure 5, HALO consists of two core components: a lightweight partition searcher and an SLO-aware stage-unified scheduler. The partition searcher addresses LoRA induced throughput degradation by finding the optimal SM partition for concurrent base-LoRA execution. The scheduler addresses SLO violations by exploiting temporal slack from heterogeneous SLOs. Specifically, it jointly manages request scheduling across both the prefill and decode stages to maximize SLO guarantees and system throughput.

The workflow begins with the offline partition searcher, which automatically locates the optimal SM partition for each base-lora layer pair at different batch sizes. Since the optimal configuration varies significantly with LoRA injection points (e.g., QKV vs. FFN), batch size, and hardware characteristics, exhaustive search is infeasible. The searcher first automatically profiles the independent execution time of two layers at different SM ratios. Then, it utilizes a feedback-driven profiling method to locate the optimal resource configuration for each base-lora layer pair.

At runtime, the SLO-aware stage-unified scheduler replaces default First-Come-First-Served policies with a deadline-aware mechanism. Upon arrival, each request is tagged with its specific SLO targets (TTFT and TBT). The scheduler continuously calculates a novel metric called request-level slack. This slack quantifies the dynamic headroom between the current time and the strict deadline of the next token to be generated. This metric unifies slack from both the prefill and decode stages, consolidating fragmented slack into actionable time slices. Requests with small slack are flagged as urgent for execution, while those with ample slack are candidates for deferral.

Leveraging this analysis, the scheduler performs decoupled batch formation. It breaks the "lock-step" constraint of traditional batching by constructing execution batches that include only urgent requests and skip non-urgent ones. This allows requests with slack to yield their compute slots to urgent requests facing imminent TTFT or TBT violations in either the prefill or decode stage. By jointly managing scheduling across both stages, the scheduler flexibly adjusts the compute allocation between decode and prefill within each iteration to maximize SLO guarantees.

Finally, the output token queue acts as a traffic shaper to decouple variable decode latency from the user experience. Generated tokens are buffered and released to the client strictly according to their TBT intervals. This smooths execution jitter, ensuring a steady stream of text. Crucially, this mechanism allows the system to "bank" time credits; if tokens are generated faster than required, the queue holds them, building a buffer that enables the scheduler to absorb future high-priority traffic bursts without violating SLOs.

Note that the base-LoRA parallelism improves performance only during the decode phase and has no effect for the prefill, since LoRA computation during prefill already achieves high utilization.

4 Lightweight Partition Searcher

4.1 Requirements and Challenges

4.1.1 Requirements. While leveraging base-LoRA layer parallelism could improve system throughput, it imposes two critical requirements on the serving system. First, the system must be capable of identifying the optimal SM partition configuration for arbitrary models under varying execution contexts. Second, the configuration search process needs to run automatically during framework initialization, which makes it essential to use an exploration method with very low overhead so that startup latency remains minimal.

4.1.2 Challenges. However, satisfying the above two requirements presents a non-trivial challenge due to the combinatorial explosion of the configuration space.

First, current LoRA services insert adapters to all possible positions (QKV, O, Gate/Up, Down), but their ranks can be chosen independently[30, 40]. Second, modern frameworks rely on CUDA graph to accelerate execution, which requires capturing a separate CUDA graph for each batch size. As a result, the optimal SM partition must be prepared for every batch size before CUDA graph capture. Third, even for a single base-LoRA layer pair, the partition search space is already very large. For example, on an NVIDIA A100 (108 SMs) with a partition granularity of 2 SMs (via GreenContext), there are $\sum_{k=0}^{54} (55 - k) = 1540$ valid configurations.

Based on this analysis, we can quantify the total profiling cost in [Equation 1](#):

$$T_{total} = N_{lora} \times N_{bs} \times N_{conf} \times T_{decode} \quad (1)$$

Here, N_{lora} denotes the number of LoRA positions, N_{bs} the number of captured batch sizes, and N_{conf} the number of candidate partitions. T_{decode} represents the profiling duration required to obtain stable measurements (≈ 100 ms). For a typical setup ($N_{lora} = 4$, $N_{bs} = 8$, $N_{conf} = 1540$), the profiling time exceeds 1.3 hour per model. This prohibitive overhead renders exhaustive search infeasible for production environments.

4.2 Feedback-based Search Method

4.2.1 Observations. To reduce the overhead described above, we attempt to design a lightweight search method that can lower the overall cost. As a first step, we conduct a series of experiments to better understand the compute behavior. These experiments lead to several key observations that guide the design.

1. SM allocation: [Figure 6](#) shows the performance of the base layer and the LoRA layer under different SM allocations. The data points are collected using *ncu* for four LoRA insertion points (QKV, O, Gate/Up, Down) in Llama3-8B with a batch size of 64 on an NVIDIA A100. All results are normalized to the best performance achieved when using all SMs. As shown, the performance gains of both two layers become very small once a certain number of SMs is allocated.

This behavior occurs because the workload of an operator is determined by the number of thread blocks, which cannot always be distributed evenly across SMs. For example, if there are 200 thread

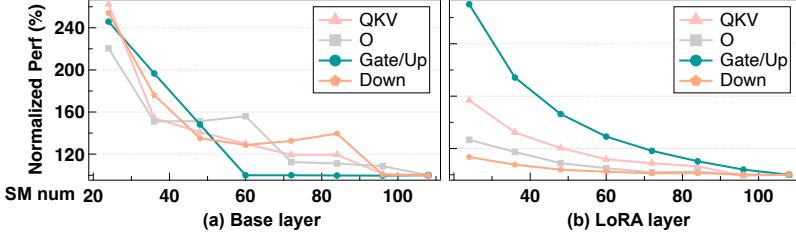


Fig. 6. The normalized performance of the base layer and the LoRA layer under different SM allocations.

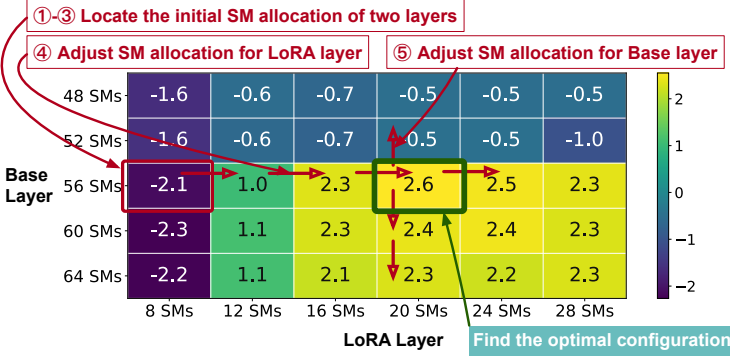


Fig. 7. The absolute performance gain (milliseconds) with different SM allocations.

blocks and 80 SMs, some SMs are assigned two thread blocks while others are assigned three, and those with three blocks determine the overall execution time. In this case, allocating 100 SMs could improve performance, whereas allocating 90 SMs would not.

Observation-1: We can allocate fewer SMs to the base layer and the LoRA layer while incurring only negligible performance degradation (within 5%).

2. SM partition: Since the command line profiling tool *ncu* cannot measure the execution time of two layers running in parallel, we can only rely on the overall decode latency to determine whether performance improves. However, *nsys* is primarily designed for visual inspection and makes it difficult to obtain fine-grained, per-layer quantitative performance metrics in a form that can be automatically processed via the command line.

Figure 7 shows the absolute latency gain (milliseconds) of Llama3-8B on an NVIDIA A100 when adjusting the SM partition between the Gate/Up layer and its LoRA layer. In this experiment, we use an input length of 128 and a batch size of 64. All data points are measured as the original sequential decode latency minus the decode latency with base-LoRA parallelism. As shown, when the Gate/Up layer is assigned 52 SMs, the decode latency even become worse. This is because the performance benefit from parallel execution is not large enough to offset the performance loss caused by reducing the resources allocated to the base layer.

Observation-2: The allocation of SMs in the base layer needs to be carefully considered, as its performance degradation may outweigh the advantage brought by base-LoRA parallelism.

3. Parallel effect: We further analyze the parallelism between the base layer and the LoRA layer in Figure 7 using the *nsys* tool. We find that none of the SM partitions in Figure 7 achieve perfect overlap between the base and LoRA layers, meaning that they do not finish at the same time. When we fix the SM allocation for the base layer and vary the allocation for the LoRA layer, or fix the

allocation for the LoRA layer and vary the allocation for the base layer, only two outcomes appear. Either the decoding latency improves or it worsens. When the decoding latency improves, the degree of overlap between the base and LoRA layers also improves. When the decoding latency worsens, the overlap becomes worse.

For example, when 56 SMs are assigned to the base layer, increasing the number assigned to the LoRA layer initially leads to a steady reduction in decoding latency because the overlap between the base and LoRA layers continues to improve. Once the number of SMs assigned to the LoRA layer exceeds 24, the latency starts to increase because the overlap between the base and LoRA layers begins to degrade, and this degradation becomes more pronounced as more SMs are added.

Observation-3: Once the SM allocation of one layer is fixed, a feedback driven search can be used to find the optimal allocation for the remaining layer.

Observation-4: The search problem with no prior knowledge over two variables (the SM allocation of two layers) can be decomposed into a two-step feedback-driven process.

4.2.2 Search Algorithms. Based on these observations, we design a feedback based search method. The method proceeds in five stages.

- (1) We use *ncu* to statistically measure the performance curves of the base layer and the LoRA layer with different SM allocations for every batch size setting.
- (2) Using these performance curves, we select the SM allocation for the base layer such that its performance drops by less than 10% from the optimum. We identify this point by collecting all performance spike points across the curves. At a spike point, such as 64 SMs for the Gate/Up layer, allocating 60 SMs leads to a performance drop of more than 10%, while increasing SMs yields no improvement of 10% or more. We find that these spikes have average improvement of 40.9%, which makes this cutoff a reliable way to preserve near optimal performance.
- (3) We choose the smallest SM number for the LoRA layer whose computation time is close to that of the base layer. When memory contention is ignored, this choice allows the two layers to finish at roughly the same time.
- (4) We fix the SM allocation for the base layer and gradually increase the allocation for the LoRA layer. As long as the decoding latency continues to improve, we keep increasing it. Once a performance drop is observed, we reverse direction and reduce the allocation until we reach the best performance point.
- (5) After fixing the SM allocation for the LoRA layer, we apply the same procedure to the base layer. We increase its allocation while decoding latency improves, and once performance degrades, we decrease it until the optimal point is found.

Figure 7 illustrates an example of this search process. As shown in the figure, the method first identifies an initial allocation of 56 SMs and 8 SMs. It then adjusts the allocation for the base layer to refine this result, followed by adjustments to the allocation for the LoRA layer to further improve performance.

Overall, this method finds a performance point within 1% of the optimal in 85.4% of cases, and in the remaining cases the performance loss is within 3%.

5 Scheduling Optimization

In this section, we present an SLO-aware stage-unified scheduler designed to address the violations caused by blind batching and maximize SLO-compliant throughput under constrained resources.

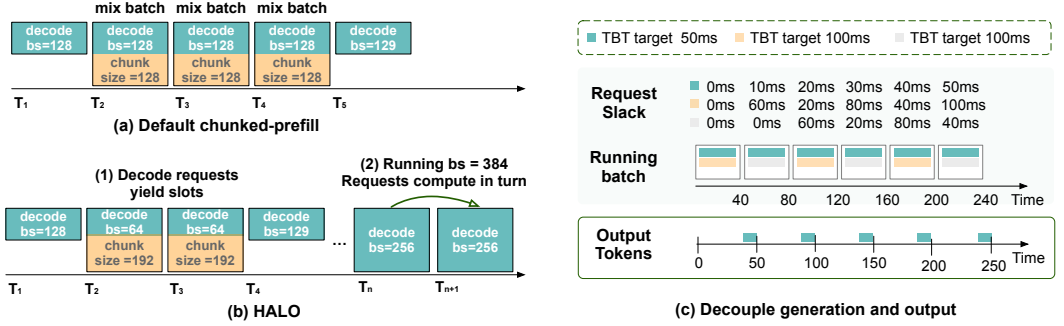


Fig. 8. The scheduling effect of chunked-prefill and HALO.

5.1 Current Workflow

Since the decode phase easily suffers from low GPU utilization due to its memory-bound nature, existing systems (SGLang and vLLM) adopt chunked-prefill proposed by Sarathi-serve to improve overall GPU utilization and system throughput. Specifically, when a new request arrives, its prefill computation is divided into multiple prefill chunks. These chunks merge with the decode batch to form a mixed batch for execution. Under the chunked-prefill mode, every iteration includes the decode batch. This causes the system to prioritize TBT targets over TTFT targets. As a result, existing systems generally set the maximum running batch size based on the strictest TBT target. This value limits both the mixed batch and the normal batch.

Figure 8(a) shows a specific scheduling example with a maximum batch size of 256. Initially, the decode batch size is 128. When $R1$ with an input length of 384 arrives, it is divided into multiple prefill chunks. The chunk size is calculated as max batch size minus decode batch size, which is 256 minus 128, giving 128. Therefore, the prefill computation of this request is divided into three prefill chunks. They participate in three decode iterations. After the three mixed batches complete, the serving system falls back to the normal decode batch with a batch size of 129.

From the above process, existing systems are unaware of heterogeneous SLOs across requests. This causes requests with relaxed targets to occupy valuable request slots, leading to SLO violations for requests with strict targets. To address this, we attempt to break the blind batching in existing systems and exploit slack from relaxed requests to improve overall SLO guarantees. However, this introduces several challenges.

5.2 Challenges for Efficient Scheduling

First, the slack of a request is generated per token, but there is not always a need to exploit it due to the fluctuated workload. However, once a request exits, its accumulated slack becomes meaningless. More importantly, directly exploiting fragmented slack introduces significant scheduling overhead. Therefore, how to efficiently utilize fragmented slack is the first challenge.

Second, each request has two stages: prefill and decode. When requests cannot enter the decode batch in time, they first suffer TTFT violations. This introduces the second challenge: how to jointly manage compute allocation across both stages. In addition, breaking blind batching also allows the system to exceed the preset maximum batch size. However, increasing running capacity without limits will eventually cause TBT violations. This introduces the third challenge: how to set the maximum running capacity to guarantee that TBT is not violated.

5.3 Decoupling Token Generation and Token Output

To solve the first challenge, we first decouple token generation from token output. Token generation is only responsible for producing tokens. Token output is responsible for returning tokens to the client side according to the TBT target. Further, we define a new metric for each request denoted as $Slack_{req}$. This metric represents how long a request can be deferred without participating in decoding. Every time a request completes one token computation, HALO updates the $Slack_{req}$ value for every request using Equation 2. In this equation, T_{decode} is the current decoding time, T_{mix} is the time for the mix batch. Here N denotes the number of elapsed mixed-batch iterations since the request arrived.

$$Slack_{req} += (TBT - T_{decode}) \quad \text{for Decode} \quad (2)$$

$$Slack_{req} += (TTFT - N * T_{mix}) \quad \text{for Prefill} \quad (3)$$

Based on these two designs, we can accumulate slack from relaxed requests during low load periods. When a bursty load arrives, these requests can yield compute resources for an extended period. However, when a request finishes its computation and leaves the current decode batch, its $Slack_{req}$ is reset to zero. This means the computational margin it has accumulated no longer contributes to improving overall system throughput. As a result, continuously increasing the headroom of requests with relaxed SLOs is not optimal.

To maximize the benefit of accumulated computational margin and to minimize wasted margin, we use the average difference in $Slack_{req}$ across all requests as an evaluation metric. Under this criterion, when any request exits, the wasted computational margin is minimized in the system.

5.4 Stage-Unified Management

After solving the first challenge, we propose stage unified management to address the second challenge. First, we derive an observation from managing the standard deviation of $Slack_{req}$ across requests. We should consider how to minimize the standard deviation of $Slack_{req}$ before a request enters computation. Therefore, when a request arrives, its initial $Slack_{req}$ is zero. It should complete its computation as quickly as possible. By finishing computation sooner, we can reach an equilibrium of $Slack_{req}$ across all requests faster.

Therefore, whenever a new request arrives, we should yield slots from requests in the current decode batch that have available slack. These requests stop computation temporarily to help the new request finish its prefill computation as quickly as possible. For example, when multiple requests have accumulated slack of N decode iterations, they should yield their request slots for the next N iterations to accelerate prefill completion.

Meanwhile, breaking blind batching removes the default maximum batch size constraint. However, continuously increasing running capacity will eventually cause TBT violations. To avoid TBT violations, similar to the default scheduling policy, we need to calculate the maximum capacity under the current scheduling method. We find that we can compute the actual capacity usage based on TBT targets. For example, if the strictest TBT target is 50ms, a request with a 100ms target only participates in computation once every two iterations. This means its actual capacity usage is 0.5.

Based on this calculation method, we can compute the actual capacity usage of all requests and ensure it remains below the originally set maximum batch size. This guarantees that HALO does not violate TBT targets. Finally, we apply the EDF (earliest deadline first) strategy to sort queued requests. This further reduces TTFT violations.

Table 3. The models and corresponding GPU usage.

Model	Llama3-8B	Yi1.5-34B	Qwen2.5-7B	Qwen2.5-32B
GPU	A100 (80G) * 1	A100 (80G) * 4	A100 (80G) * 1	A100 (80G) * 4

Table 4. The hardware and software specifications.

CPU	GPU	OS	Driver	CUDA Runtime	Serving Framework
AMD EPYC 7513	A100, Ada6000	Ubuntu 22.04	570.144	CUDA 12.9	SGLang v0.5.4

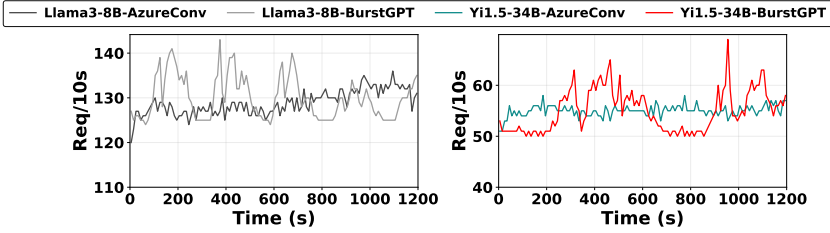


Fig. 9. The two real-world workload traces after scaling in hospital Scenario.

6 Evaluation

6.1 Experiment Setup

6.1.1 Testbed. We evaluate HALO on large language models of different scales under both single GPU and multi GPU settings to demonstrate its effectiveness and generality. Table 3 summarizes the evaluated models and their corresponding hardware configurations. As shown, we use a single A100 GPU to serve Llama3-8B[13] and Qwen2.5-7B[34]. For larger models, including Yi1.5-34B[2] and Qwen2.5-32B[34], we use a server equipped with four A100 SXM4 GPUs with 80 GB of memory each. These GPUs are connected through NVLink. Table 4 lists the software configurations used in our experiments, which are kept consistent across all models.

Note that, Yi1.5-34B[2] is built on the Llama model architecture as there is no official Llama3-30B. In addition, we also evaluate the HALO on the Nvidia Ada6000 to demonstrate its effectiveness.

6.1.2 Workloads. Following prior works[31, 38], we generate workloads using request patterns derived from production traces, since these traces reflect real world LLM usage. At the same time, we used the input length and output length distributions from the dataset, because the original LLM traces are not collected from an on-premise deployment scenario.

6.1.3 Metrics. HALO focuses on system throughput under guaranteed SLOs. Following prior works[1, 26, 28, 43], we use tail latency such as P99 to assess SLO attainment. Meanwhile, there is another metric for the decode phase: TPOT (time per output token). In comparison, TBT accounts for the latency of each individual token, whereas TPOT is an average metric that may mask poor performance of some tokens [42]. Thus, we choose TBT over TPOT as a stricter SLO metric. Table 1 and Table 2 show all SLO setups for all models.

6.1.4 Baselines. We compare HALO with four baselines. All baselines run in the chunked-prefill mode proposed by Sarathi-Serve[1].

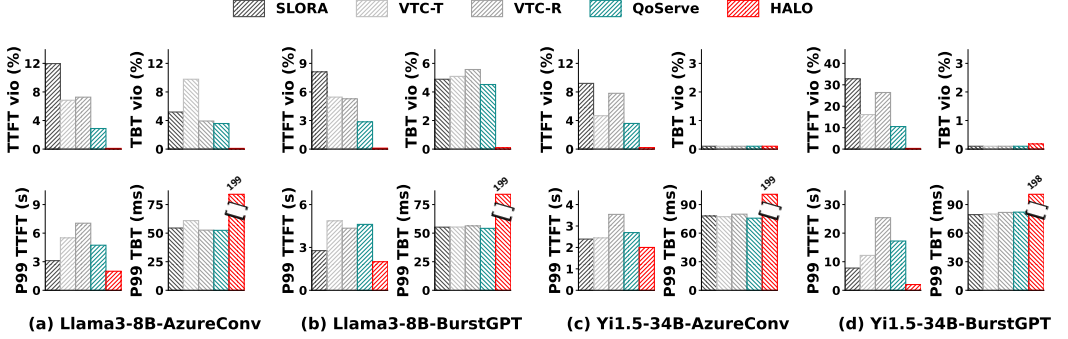


Fig. 10. P99 TTFT and TBT for Llama3-8B and Yi1.5-34B on real-world AzureConv and BurstGPT workloads.

- **SLoRA**: SLoRA[31] is a state-of-the-art multi-LoRA serving system that batches the computation of multiple LoRA adapters and uses a first come first served (FCFS) scheduling policy.
- **VTC-T**: VTC[32] targets fairness in multi-tenant LLM serving based on token-level metrics. We use VTC-T to denote the original VTC design. It enforces fairness across four services by equalizing the number of generated tokens.
- **VTC-R**: VTC-R is a VTC variant that enforces fairness using the per-service TTFT (first-token) violation rate, defined as the number of requests whose first token misses the TTFT target divided by the total number of requests of that service.
- **QoServe**: QoServe[12] is another state-of-the-art multi-LoRA serving system. It adopts the earliest deadline first strategy under high load and dynamically adjusts the chunk size based on the strictest TBT target in the batch.

6.2 Evaluation in Hospital Scenario

We first evaluate HALO using Llama3-8B and Yi1.5-34B on real-world workload traces. Figure 9 shows their request rates after scaling down, since they originate from a large cluster. As shown in the figure, AzureConv is relatively stable while BurstGPT exhibits bursty request patterns.

Figure 10 shows the latency distribution for TTFT and TBT in the *Hospital* scenario. HALO improves P99 TTFT (99%-ile TTFT) by 2.1 $\times$, 3.1 $\times$, 5.0 $\times$, and 3.6 $\times$ over SLoRA, VTC-T, VTC-R, and QoSServe respectively. HALO reduces TTFT violations by 15.3%, 8.1%, 11.6%, and 4.8% compared to SLoRA, VTC-T, VTC-R, and QoSServe. HALO also reduces TBT violations by 2.5%, 3.7%, 2.3%, and 2.0% compared to SLoRA, VTC-T, VTC-R, and QoSServe.

The performance improvement comes from two key factors. First, HALO accelerates LoRA execution by enabling parallel execution of the base layer and the LoRA layer. In contrast, all baselines suffer from the inefficiencies of Punica, which limits end-to-end latency improvements. By using base-LoRA parallelism, HALO reduces decode latency and directly raises the upper bound on system throughput.

Second, while the baselines can distinguish requests by service type, they cannot exploit SLO differences across requests. HALO explicitly leverages request-level slack and SLO differences to guide scheduling decisions. Its SLO-aware scheduling strategy uses accumulated slack to enable more flexible scheduling. Specifically, HALO returns each token according to its SLO target. This introduces higher P99 TBT but improves both TTFT and TBT guarantees.

Note that, for the baselines, Llama3-8B and Qwen2.5-7B suffer from severe TBT violations, while Yi1.5-34B and Qwen2.5-32B do not. This is because the maximum batch size for Llama3-8B and Qwen2.5-7B is set to 384. Their decode time at 1K sequence length is 45 to 46ms. When the sequence

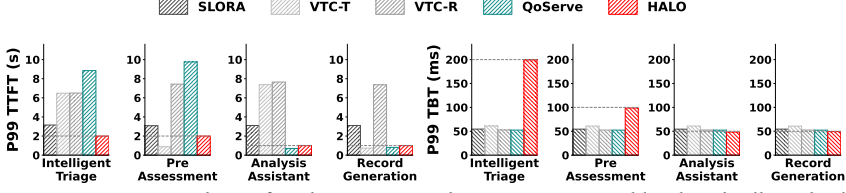


Fig. 11. P99 TTFT and TBT for Llama3-8B on the AzureConv workload with all methods.

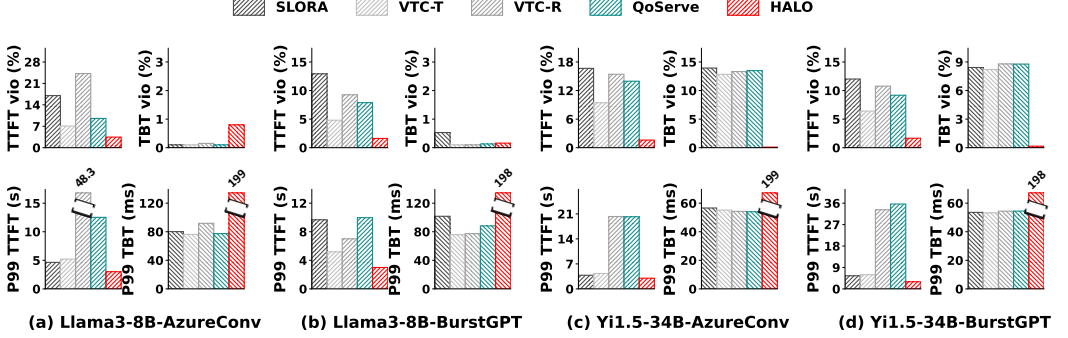


Fig. 12. P99 TTFT and TBT for Llama3-8B and Yi1.5-34B on real-world AzureConv and BurstGPT workloads.

length increases further, they easily suffer TBT violations. The maximum batch size for Yi1.5-34B and Qwen2.5-32B is set to 256. Their decode time at 1K sequence length is 70 to 73ms. Even when the sequence length increases further, they do not suffer TBT violations. In contrast, HALO exploits TBT slack to coordinate the computation order. This allows it to avoid all TBT violations across every model.

Figure 11 shows the P99 TTFT and P99 TBT for four services. As shown, the two services with strict SLOs (analysis and record) suffer severe TBT violations, while the two services with relaxed SLOs (triage and preconsult) have their TBT targets fully met. This further demonstrates that prior works cannot exploit TBT slack. Also, it is worth noting that the P99 TBT for the two strict SLO services is 70ms against a TBT target of 50ms. This is because the TBT target is set based on a 1K context length, while actual requests may have a 4K context length. If we set a smaller chunk size based on 4K context length, TTFT violations will increase dramatically. In contrast, since HALO performs SLO-aware scheduling, it avoids TBT violations caused by varying context lengths.

For VTC-T and VTC-R, they can only control the order in which requests enter the decode batch. When VTC-T and VTC-R adjust the request serving order based on TTFT violations, they can achieve a fair TTFT violation rate across the four services. However, they cannot resolve TTFT violations and TBT violations, since they are unaware of TBT target differences across requests.

For QoServe, it uses the earliest deadline first strategy to adjust the order in which requests enter the batch. This reduces TTFT violations to some extent. However, it also cannot exploit the SLO slack from TBT differences. Although QoServe can dynamically adjust the chunk size based on the strictest TBT target in the running batch, all four services in this scenario are user-facing services. There is little opportunity for QoServe to benefit from adjusting the chunk size.

6.3 Evaluation in Service Center Scenario

Figure 12 shows the latency distribution for TTFT and TBT in the *Service Center Scenario*. HALO improves P99 TTFT (99%-ile TTFT) by 1.9 $\times$, 1.7 $\times$, 9.1 $\times$, and 6.5 $\times$ over SLORA, VTC-T, VTC-R, and

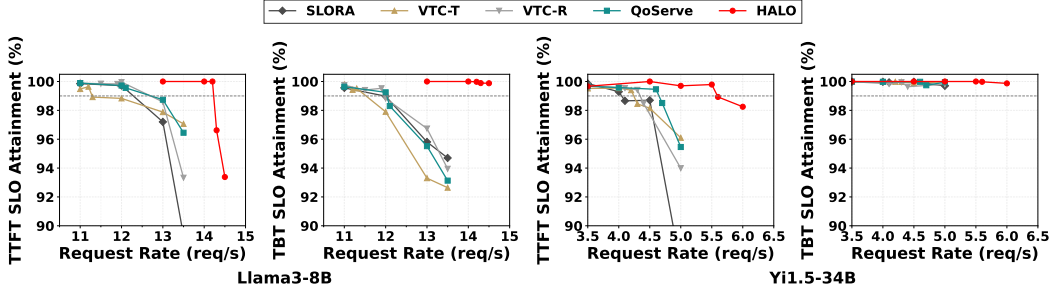


Fig. 13. SLO Attainment with Different RPS for Llama3-8B and Yi1.5-34B in hospital Scenario.

QoServe respectively. HALO reduces TTFT violations by 12.6%, 4.8%, 12.8%, and 8.1% compared to SLoRA, VTC-T, VTC-R, and QoServe. HALO also reduces TBT violations by 5.4%, 5.0%, 5.2%, and 5.3% compared to SLoRA, VTC-T, VTC-R, and QoServe.

Although the hospital and service center scenarios involve different LLM services, the results above show that HALO consistently achieves strong performance gains in both settings. This demonstrates that HALO can effectively support diverse on-premise LLM deployment scenarios, even when they differ in TTFT and TBT targets, request characteristics, and underlying datasets.

6.4 SLO Attainment with Different RPS

We also evaluate the SLO attainment of different systems under varying RPS. Following prior works [1, 26, 28, 31], the request arrival pattern at each RPS follows a Poisson distribution. Four types of requests are generated with equal probability using the corresponding dataset and SLO targets from the hospital scenario. We stop testing once the serving system becomes unstable or fails to meet the TTFT target.

Figure 13 shows the SLO attainment results under gradually increasing workloads. Under the constraint of meeting P99 TTFT guarantees, HALO achieves 29.2%, 29.5%, 24.8%, and 19.5% throughput improvement compared to SLoRA, VTC-T, VTC-R, and QoServe. HALO still provides P99 TTFT and P99 TBT guarantees at higher RPS. This again comes from two factors: base-LoRA parallelism and SLO-aware scheduling.

Specifically, as shown in Figure 13, all baselines begin to suffer TTFT violations once RPS exceeds 13. This is because they cannot exploit TBT slack to enable earlier prefill computation. Meantime, all baselines also suffer some TBT violations, even though chunked-prefill sets the maximum batch size to prioritize decode computation. This is because long context requests (like 4K length) cause some decode iterations to exceed the TBT target (set based on 1K length). In contrast, HALO exploits TBT slack to serve newly arrived requests faster and reduce TTFT violations. It also uses TBT slack to mitigate TBT violations caused by long context lengths.

6.5 Different LLM models

Beyond the Llama series, we also select models from the Qwen series for evaluation. Specifically, we choose Qwen2.5-7B and Qwen2.5-32B. Figure 14 shows the SLO attainment results under gradually increasing workloads. Under the constraint of meeting P99 TTFT guarantees, HALO achieves 27.2%, 32.2%, 25.0%, and 21.8% throughput improvement compared to SLoRA, VTC-T, VTC-R, and QoServe. These results show that HALO also achieves significant performance improvement on new models. This is because all designs in HALO are model-agnostic. Its designs are rooted in base-LoRA layer parallelism and SLO-aware scheduling, which are independent of model architecture.

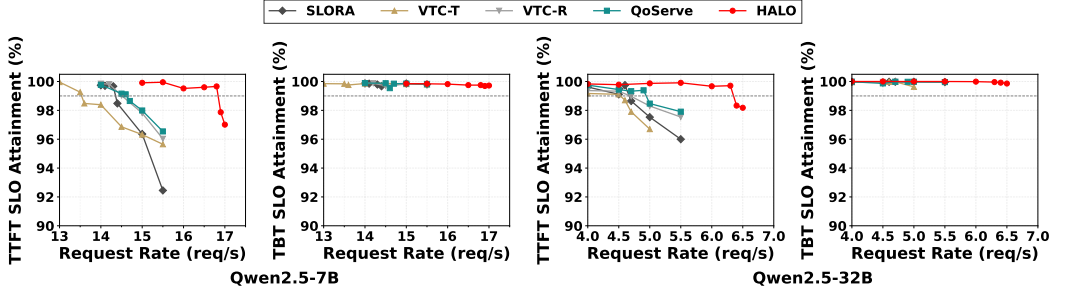


Fig. 14. SLO Attainment with Different RPS for Qwen2.5-7B and Qwen2.5-32B in hospital Scenario.

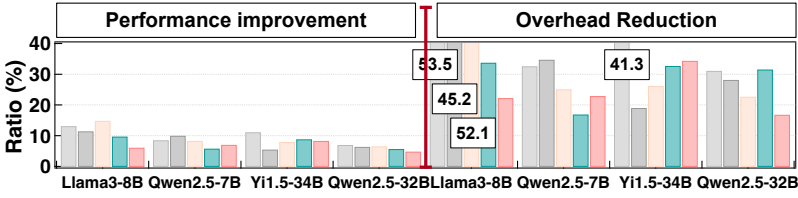


Fig. 15. The performance improvement and overhead reduction brought the base-lora parallelism.

6.6 The Effect of Base-LoRA Parallelism

Figure 15 shows the end-to-end decode latency improvement and the reduction in LoRA-induced overhead for four models under different batch sizes. As shown in the figure, when the sequence length is 128, base-LoRA parallelism achieves an average performance improvement of 8.3%, with a maximum improvement of 14.7%. It reduces LoRA-induced execution overhead by 31.1% on average and by up to 53.5% at maximum. These results demonstrate the effectiveness of base LoRA parallelism.

When the sequence length increases to 1024, the average performance improvement decreases to 6.4%. This reduction occurs because, although base-LoRA parallelism itself is insensitive to sequence length, another major component of inference, attention computation, increases with sequence length. As the baseline decode latency grows, the relative benefit of base-LoRA parallelism becomes smaller. We also note that prefill is computationally intensive, and base-LoRA parallelism does not improve performance during the prefill phase.

6.7 Ablation Study

In this experiment, we conduct ablation studies to understand the contribution of each component of HALO. We construct HALO-P, which enables only base-LoRA parallelism, and HALO-S, which enables only SLO-aware scheduling. The experiment is performed on Llama3-8B using a single A100 GPU.

Figure 16(a) shows the latency distribution for TTFT and TBT in the Hospital Scenario. HALO achieves 18.3%, 4.1%, and 10.3% throughput improvement compared to SLORA, HALO-P, and HALO-S, respectively. The results show that enabling only base-LoRA parallelism still leads to significant SLO violations. In contrast, enabling only SLO-aware scheduling is constrained by sequential base-LoRA execution. These findings indicate that both components are necessary to achieve the performance benefits of HALO.

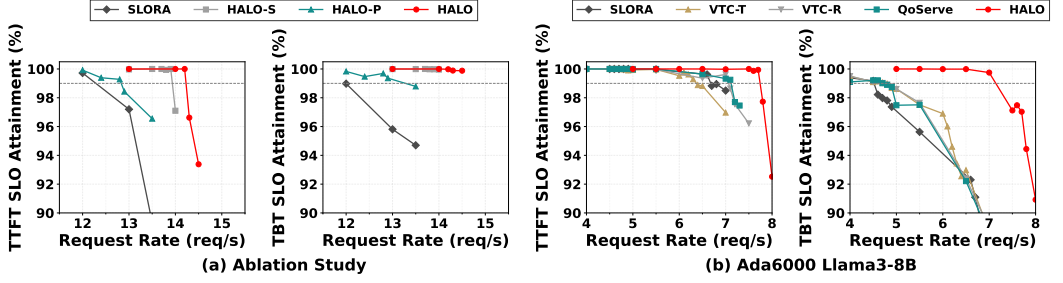


Fig. 16. The throughput and violation in hospital Scenario: (a) SLORA, HALO-P, HALO-S and HALO; (b) SLORA, VTC-T, VTC-R, QoServe and HALO for Llama3-8B on the Ada6000.

6.8 Different GPU

In this experiment, we evaluate HALO on a different GPU to verify its effectiveness across hardware platforms. Specifically, we run Llama3-8B on a single Ada6000 GPU.

Figure 16(b) shows the TTFT and TBT latency distributions in the hospital scenario. Under the constraint of meeting P99 TTFT guarantees, HALO achieves 23.0%, 27.4%, 19.9%, and 19.5% throughput improvement compared to SLORA, VTC-T, VTC-R, and QoSServe. The throughput improvement comes from both TTFT and TBT optimization. As shown in the figure, all baselines suffer from severe TBT violations. This is because Llama3-8B with a batch size of 256 at sequence length 256 requires 49ms for decode computation. When the input length exceeds 256, the baselines suffer from severe TBT violations. On the other hand, if we set the maximum batch size below 256 such as 128, the decode computation encounters severe GPU underutilization. The results show that HALO continues to deliver strong performance and efficiently support GPUs beyond the A100.

6.9 Overhead

HALO’s overhead primarily comes from two sources. Firstly, HALO needs to determine the base-LoRA partition during the serving startup phase. Secondly, during runtime scheduling, HALO needs to decide which requests enter the running batch.

For the partition search, the process requires 12.4 profiling runs on average. For a typical setup ($N_{lora} = 4, N_{bs} = 8, N_{conf} = 12.4, T_{decode} = 100ms$), the profiling time exceeds 39.7 seconds per model, which is tolerable for the framework startup. For the runtime scheduling, the sorting process on the CPU side is implemented using an asynchronous way, which involves no scheduling overhead.

7 Related works

LLM serving optimization: LLM serving optimization has attracted great attention. Existing work explores several directions, including compute management[1, 26, 43], memory management[16, 21, 42], and GPU kernel optimization[8, 39]. Splitwise[26] disaggregates LLM serving into separate prefill and decode instances, while chunked prefill[1] divides the prefill phase into smaller chunks and fuses each chunk with a decode iteration. PagedAttention[16] introduces a paged memory pool that enables reuse of key value caches across the prefill and decode phases. SGLang[42] exploits context aware caching to maximize reuse of key value segments across requests. FlashAttention[8] and FlashInfer[39] focus on designing specialized kernels to accelerate attention computation. These approaches target general improvements to the serving framework, and are orthogonal to the techniques proposed in HALO.

LoRA serving optimization: Existing works[6, 19, 27, 31, 38, 40] on LoRA serving optimization largely fall into two categories. One line of work targets fast switching among multiple LoRAs, while the other focuses on reducing LoRA execution latency. SLoRA[31] enables scalable serving of thousands of adapters by introducing unified paging. LoRAX[27] supports just in time dynamic loading of adapters through a tiered caching (GPU, CPU, and disk). CaraServe[19] concentrates on the synchronization between CPU and GPU to hide the latency of fetching adapters. These systems are designed to handle scenarios where thousands of LoRA adapters. In contrast, an on-premise deployment typically supports only a small number of LoRA adapters, which makes these approaches ineffective. In addition, Punica[6] proposes a new CUDA kernel to accelerate LoRA execution. However, our results show that this optimization is still far from optimal when considering end-to-end latency.

LLM scheduling strategy: Recent researches[7, 15, 32, 37] have devoted substantial effort to LLM scheduling strategies that aim to minimize average job completion time. LoRAServe[15] applies dynamic adapter placement and request routing to balance load across multiple GPUs. FastServe[37] uses preemptive scheduling to mitigate head of line blocking caused by long running requests. VTC[32] introduces a deficit round robin scheduler based on virtual tokens to provide fairness in LLM serving. SLOs-Serve[7] supports multiple SLOs and multi-instance deployment through dynamic request routing. While these approaches can reduce job completion time in certain scenarios, they do not address multi-SLO optimization in the on-premise LLM deployment. This is because the on-premise deployment generally has a single instance and tightly constrained computational resources.

8 Conclusion

In this paper, we present HALO, a specialized scheduling framework that addresses the performance bottlenecks of LoRA-enabled on-premise LLM servers in privacy-sensitive sectors. By introducing a spatial multiplexing strategy that partitions GPU Streaming Multiprocessors to overlap Base and LoRA computations, and an SLO-aware scheduler driven by "request-level slack," we successfully decouple execution from the constraints of rigid batching. Our evaluation demonstrates that HALO effectively mitigates resource contention and significantly reduces SLO violations while improving aggregate throughput compared to state-of-the-art baselines, providing a robust foundation for secure, high-performance local LLM inference.

Acknowledgments

This work is partially sponsored by the National Natural Science Foundation of China (62302302, 62232011), and Natural Science Foundation of Shanghai Municipality (24ZR1430500). This work was supported by Ant Group through CCF-Ant Research Fund. Han Zhao is the corresponding author.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [2] 01. AI, :, Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, Kaidong Yu, Peng Liu, Qiang Liu, Shawn Yue, Senbin Yang, Shiming Yang, Tao Yu, Wen Xie, Wenhao Huang, Xiaohui Hu, Xiaoyi Ren, Xinyao Niu, Pengcheng Nie, Yuchi Xu, Yudong Liu, Yue Wang, Yuxuan Cai, Zhenyu Gu, Zhiyuan Liu, and Zonghong Dai. 2024. Yi: Open Foundation Models by 01.AI. arXiv:2403.04652 [cs.CL]
- [3] anon8231489123. [n. d.]. Dataset-ShareGPT. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. Accessed: 2026-01-27.

- [4] Ondrej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Aleksandra Tamchyna. 2014. Findings of the 2014 Workshop on Statistical Machine Translation. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*. Association for Computational Linguistics, Baltimore, Maryland, USA, 12–58. <http://www.aclweb.org/anthology/W/W14/W14-3302>
- [5] Michael L Burns, Ssu-Ying Chen, Chu-An Tsai, John Vandervest, Balaji Pandian, Paige Nong, David A Hanauer, Andrew Rosenberg, and Jody Platt. 2025. Generative AI costs in large healthcare systems, an example in revenue cycle. *NPJ digital medicine* 8, 1 (2025), 579.
- [6] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2024. Punica: Multi-tenant lora serving. *Proceedings of Machine Learning and Systems* 6 (2024), 1–13.
- [7] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B Gibbons. 2025. SLOs-Serve: Optimized Serving of Multi-SLO LLMs. *arXiv preprint arXiv:2504.08784* (2025).
- [8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [9] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL] <https://arxiv.org/abs/2501.12948>
- [10] DELL. [n. d.]. The Dell AI Factory with NVIDIA: Accelerating AI Adoption for the Enterprise. <https://www.nvidia.com/en-us/on-demand/session/aisummit24-si1250/>. Accessed: 2026-01-27.
- [11] Othmane Friha, Mohamed Amine Ferrag, Burak Kantarci, Burak Cakmak, Arda Ozgun, and Nassira Ghoualmi-Zine. 2024. Llm-based edge intelligence: A comprehensive survey on architectures, applications, security and trustworthiness. *IEEE Open Journal of the Communications Society* (2024).
- [12] Kanishk Goel, Jayashree Mohan, Nipun Kwatra, Ravi Shreyas Anupindi, and Ramachandran Ramjee. 2026. QoServe: Breaking the Silos of LLM Inference Serving. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1492–1507.
- [13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [15] Shashwat Jaiswal, Shrikara Arun, Anjali Parayil, Ankur Mallick, Spyros Mastorakis, Alind Khare, Chloi Alverti, Renee St Amant, Chetan Bansal, Victor Rühle, et al. 2025. Serving Heterogeneous LoRA Adapters in Distributed LLM Inference Systems. *arXiv preprint arXiv:2511.22880* (2025).
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [17] Lavita. [n. d.]. Dataset-ChatDoctor. <https://huggingface.co/datasets/lavita/ChatDoctor-HealthCareMagic-100k>. Accessed: 2026-01-27.
- [18] Borui Li, Yitao Wang, Haoran Ma, Ligeng Chen, Jun Xiao, and Shuai Wang. 2025. MobiLoRA: Accelerating LoRA-Based LLM Inference on Mobile Devices via Context-Aware KV Cache Optimization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 23400–23410.
- [19] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. 2024. Caraserve: Cpu-assisted and rank-aware lora serving for generative llm inference. *arXiv preprint arXiv:2401.11240* (2024).
- [20] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, et al. 2025. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. *arXiv preprint arXiv:2501.12162* (2025).
- [21] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [22] Malikeh1375. [n. d.]. Dataset-MedicalData. <https://huggingface.co/datasets/Malikeh1375/medical-question-answering-datasets>. Accessed: 2026-01-27.
- [23] Nestor Maslej, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, Njenga Kariuki, Emily Capstick, Anka Reuel, Erik Brynjolfsson, John Etchemendy, et al. 2025. Stanford HAI AI Index Report 2025. *arXiv preprint arXiv:2504.07139* (2025).
- [24] OpenAI. 2023. GPT-4 Technical Report. <https://openai.com/index/gpt-4-research/>
- [25] Guanzhong Pan, Vishal Chodnekar, Abinas Roy, and Haibo Wang. 2025. A Cost-Benefit Analysis of On-Premise Large Language Model Deployment: Breaking Even with Commercial LLM Services. *arXiv preprint arXiv:2509.18101* (2025).

- [26] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [27] Predise. 2025. LoRAX: Multi-LoRA inference server that scales to 1000s of fine-tuned LLMs. <https://github.com/predibase/lorax>
- [28] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*. 155–170.
- [29] IDC Report. 2026. Artificial Intelligence Infrastructure Spending. <https://my.idc.com/getdoc.jsp?containerId=prUS53894425>
- [30] Zheyu Shen, Yexiao He, Ziyao Wang, Yuning Zhang, Guoheng Sun, Wanghao Ye, and Ang Li. 2025. EdgeLoRA: An Efficient Multi-Tenant LLM Serving System on Edge Devices. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 138–153.
- [31] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. 2023. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285* (2023).
- [32] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 965–988.
- [33] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [34] Qwen Team et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* (2024).
- [35] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971* [cs.CL] <https://arxiv.org/abs/2302.13971>
- [36] Alyssa Unell, Mehr Kashyap, Michael Pfeffer, and Nigam Shah. 2025. Real-World Usage Patterns of Large Language Models in Healthcare. *medRxiv* (2025), 2025–05.
- [37] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).
- [38] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. {dLoRA}: Dynamically orchestrating requests and adapters for {LoRA}-{LLM} serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 911–927.
- [39] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005* (2025).
- [40] Hang Zhang, Jiuchen Shi, Yixiao Wang, Quan Chen, Yizhou Shan, and Minyi Guo. 2025. Improving the Serving Performance of Multi-LoRA Large Language Models via Efficient LoRA and KV Cache Management. *arXiv preprint arXiv:2505.03756* (2025).
- [41] Xinlu Zhang, Chenxin Tian, Xianjun Yang, Lichang Chen, Zekun Li, and Linda Ruth Petzold. 2023. AlpaCare: Instruction-tuned Large Language Models for Medical Application. *arXiv:2310.14558* [cs.CL]
- [42] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody_Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2023. Efficiently Programming Large Language Models using SGLang. *arXiv preprint arXiv:2312.07104* (2023).
- [43] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.